

Fundamentals of algorithm analysis: Space and time complexity of an algorithm

Module I

Analyzing Algorithms

- Predict the amount of resources required:
 - **memory**: how much space is needed?
 - **computational time**: how fast the algorithm runs?
- FACT: running time grows with the size of the input
- Input size (number of elements in the input)
 - Size of an array, polynomial degree, # of elements in a matrix, # of bits in the binary representation of the input, vertices and edges in a graph

Def: Running time = the number of primitive operations (steps) executed before termination

- Primitive operations includes arithmetic operations (+, -, *), data movement, control, decision making (*if, while*) and comparison.

Algorithm Analysis: Example

- *Alg.:* MIN ($a[1], \dots, a[n]$)
 $m \leftarrow a[1];$
 for $i \leftarrow 2$ to n
 if $a[i] < m$
 then $m \leftarrow a[i];$
- **Running time:**
 - the number of primitive operations (steps) executed before termination
$$T(n) = 1 \text{ [first step]} + (n-1) \text{ [for loop]} + (n-1) \text{ [if condition]} + (n-1) \text{ [the assignment in then]} = 3n - 2$$
- **Order (rate) of growth:**
 - The leading term of the formula
 - Expresses the asymptotic behavior of the algorithm

Typical Running Time Functions

- 1 (constant running time):
 - Instructions are executed once or a few times
- $\log N$ (logarithmic)
 - A big problem is solved by cutting the original problem in smaller sizes, by a constant fraction at each step
- N (linear)
 - A small amount of processing is done on each input element
- $N \log N$
 - A problem is solved by dividing it into smaller problems, solving them independently and combining the solution

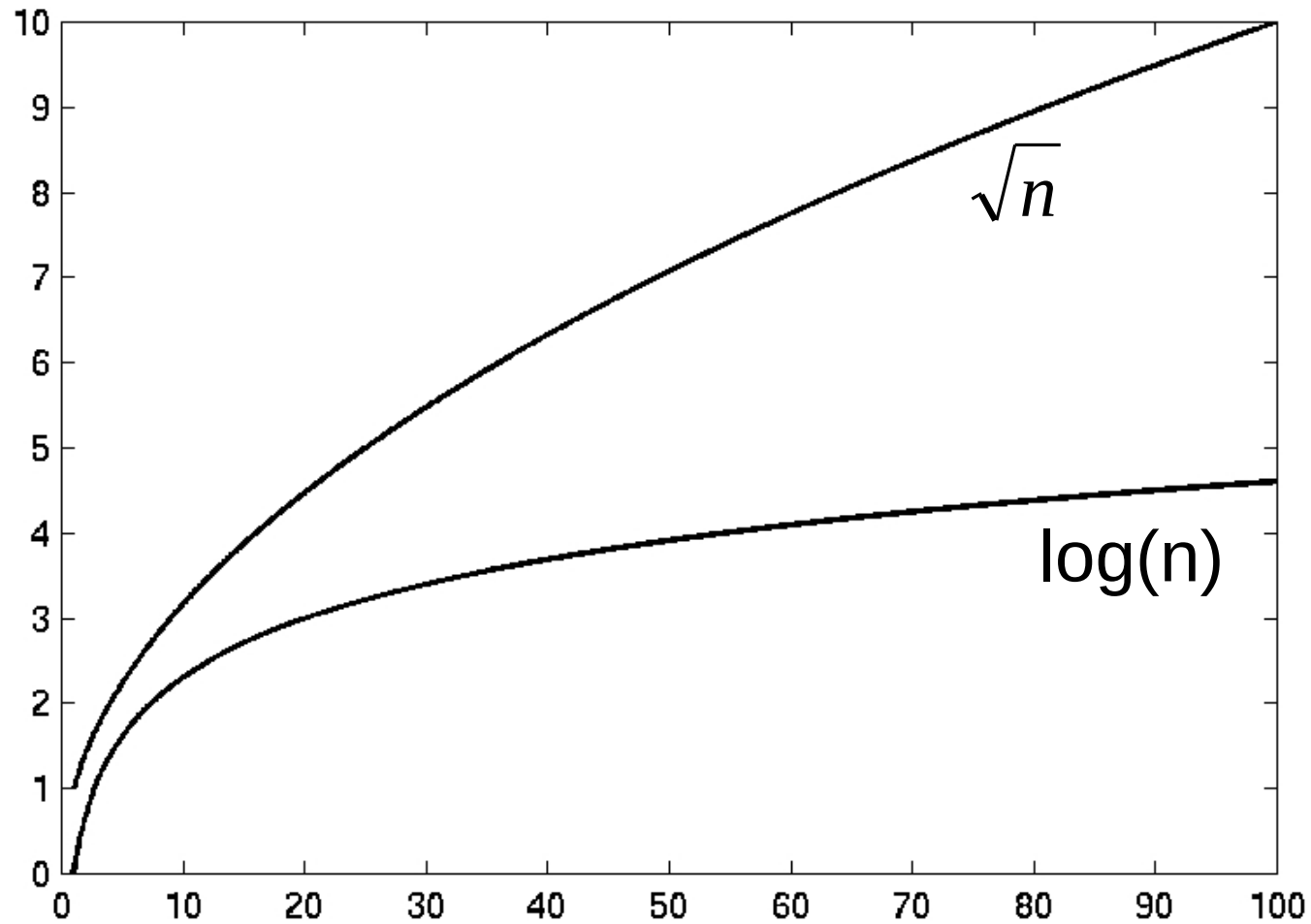
Typical Running Time Functions

- N^2 (quadratic)
 - Typical for algorithms that process all pairs of data items (double nested loops)
- N^3 (cubic)
 - Processing of triples of data (triple nested loops)
- N^K (polynomial)
- 2^N (exponential)
 - Few exponential algorithms are appropriate for practical use

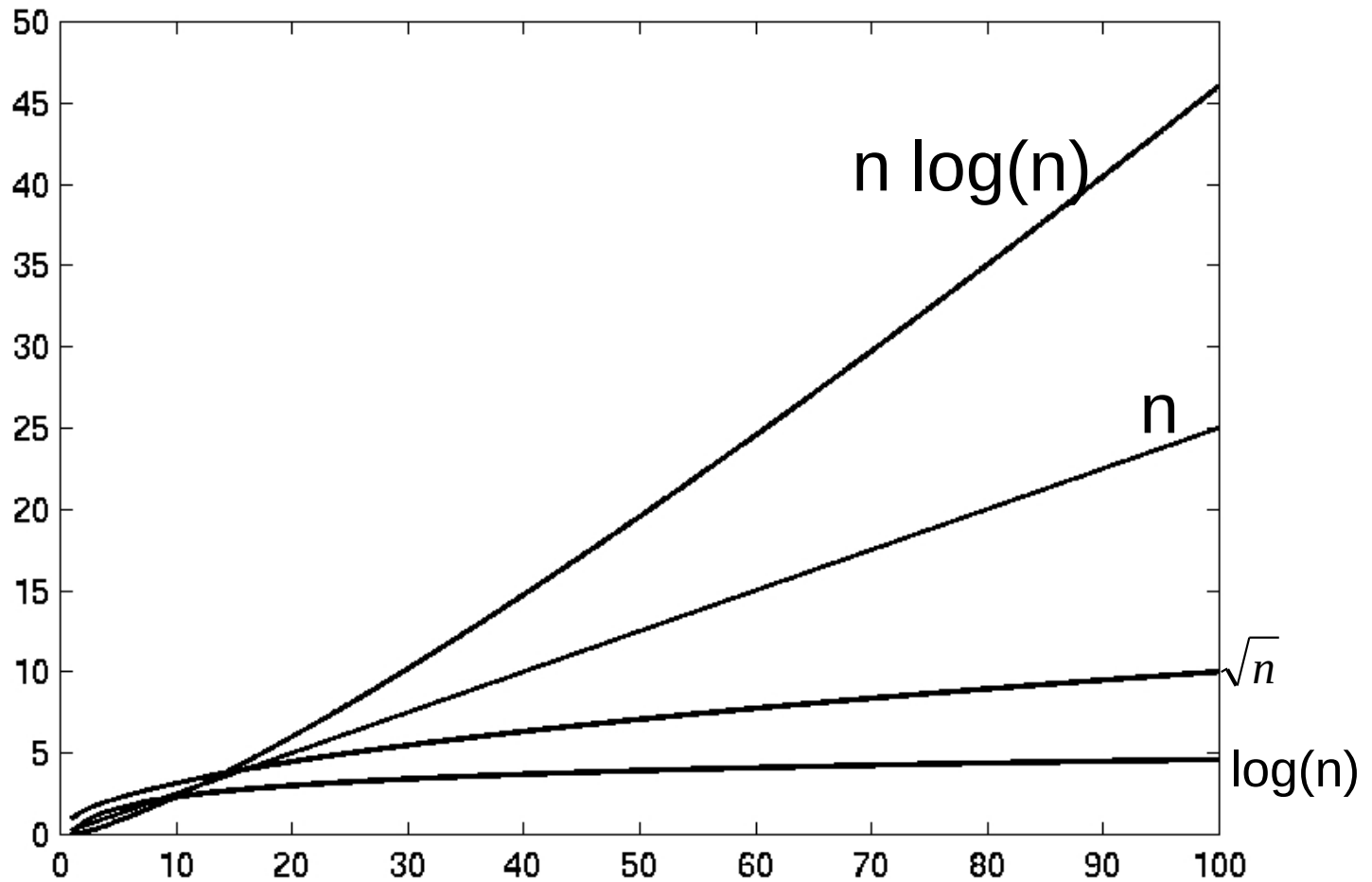
Growth of Functions

n	1	lgn	n	n lgn	n²	n³	2ⁿ
1	1	0.00	1	0	1	1	2
10	1	3.32	10	33	100	1,000	1024
100	1	6.64	100	664	10,000	1,000,000	1.2×10^{30}
1000	1	9.97	1000	9970	1,000,000	10^9	1.1×10^{301}

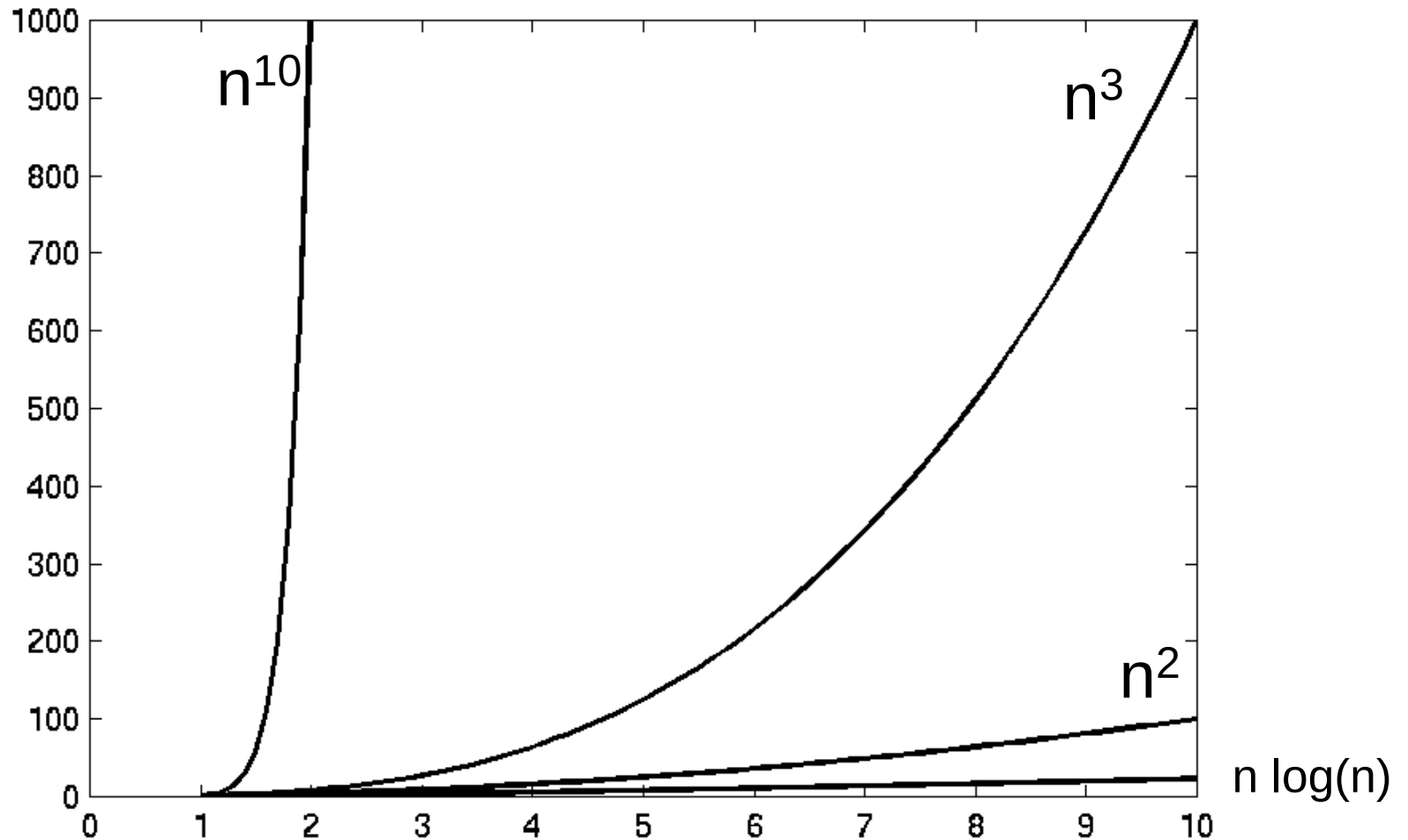
Complexity Graphs



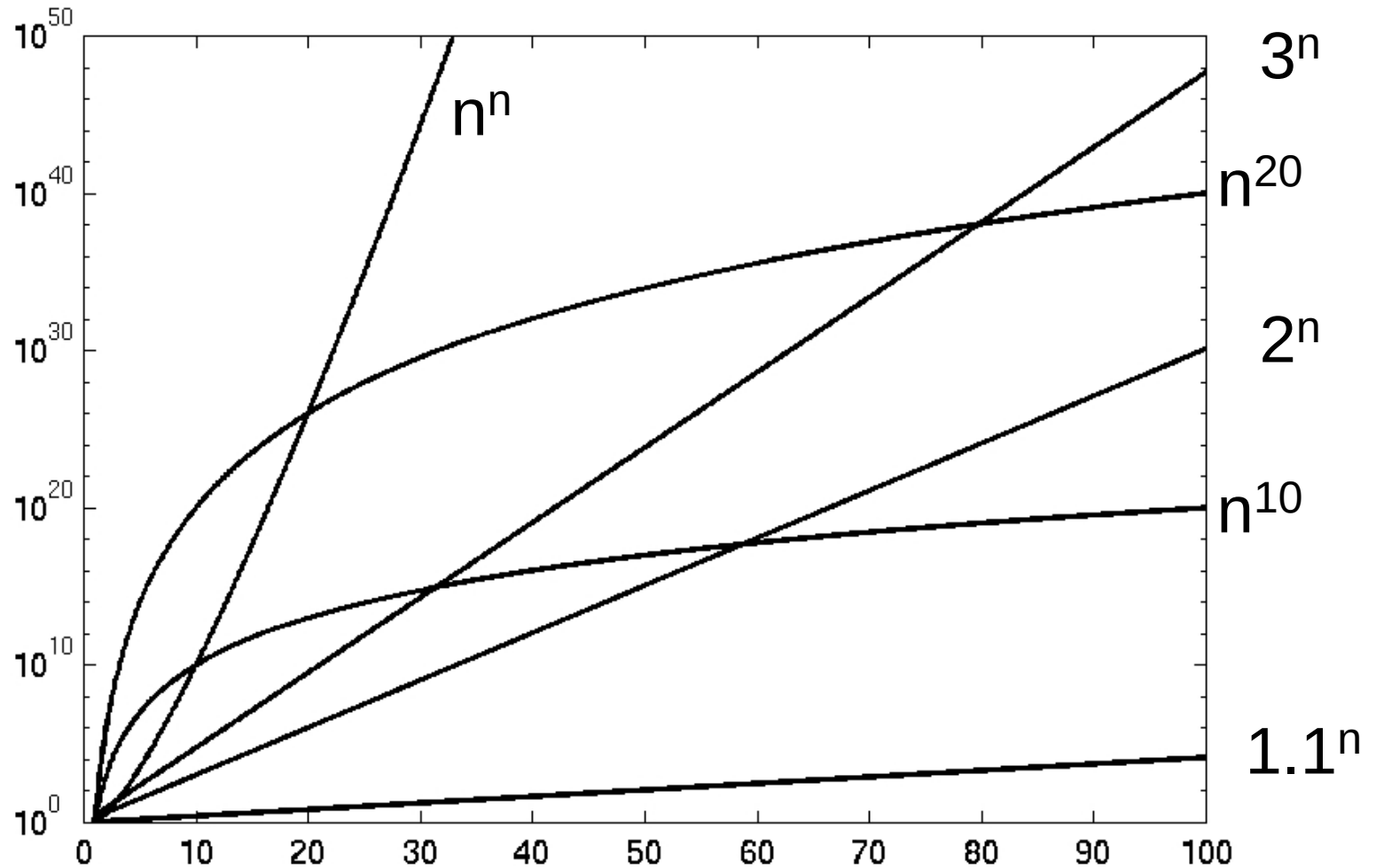
Complexity Graphs



Complexity Graphs



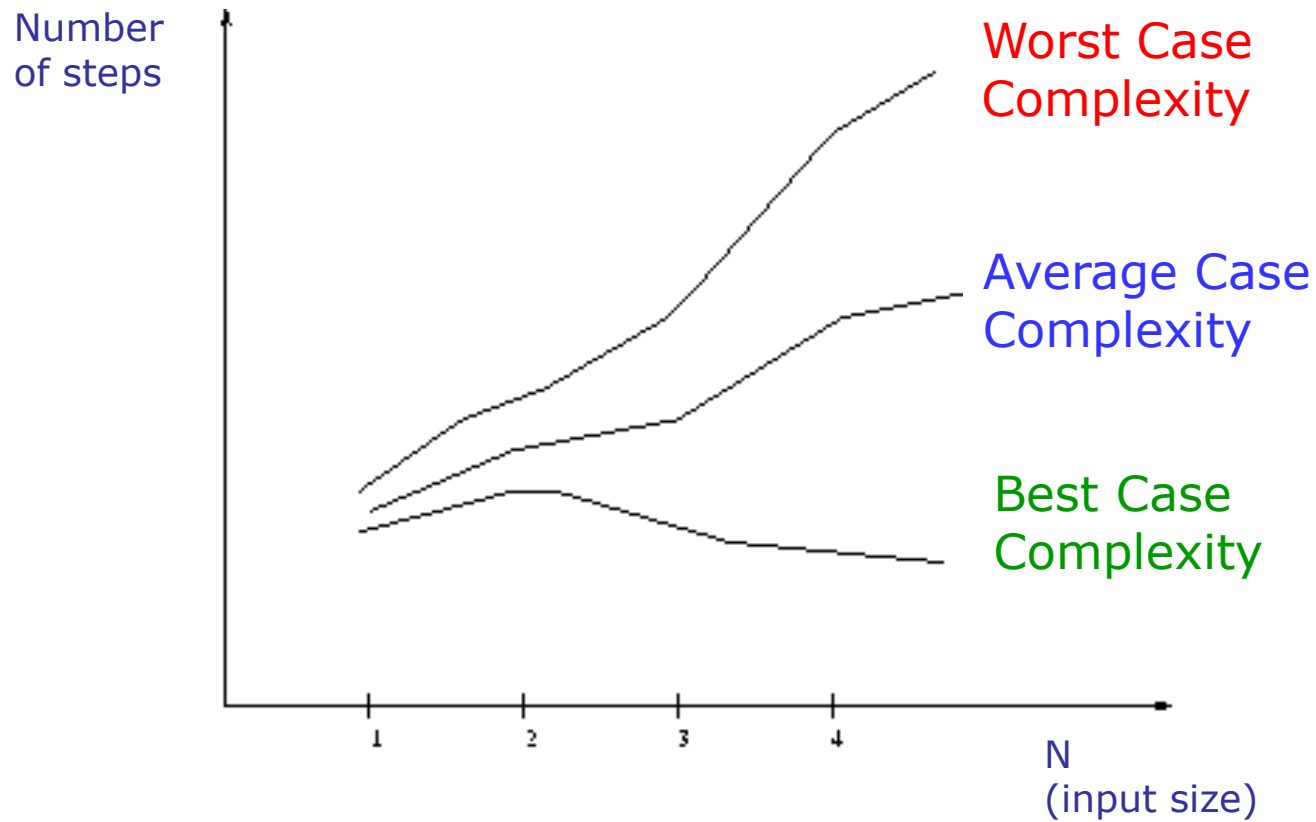
Complexity Graphs (log scale)



Algorithm Complexity

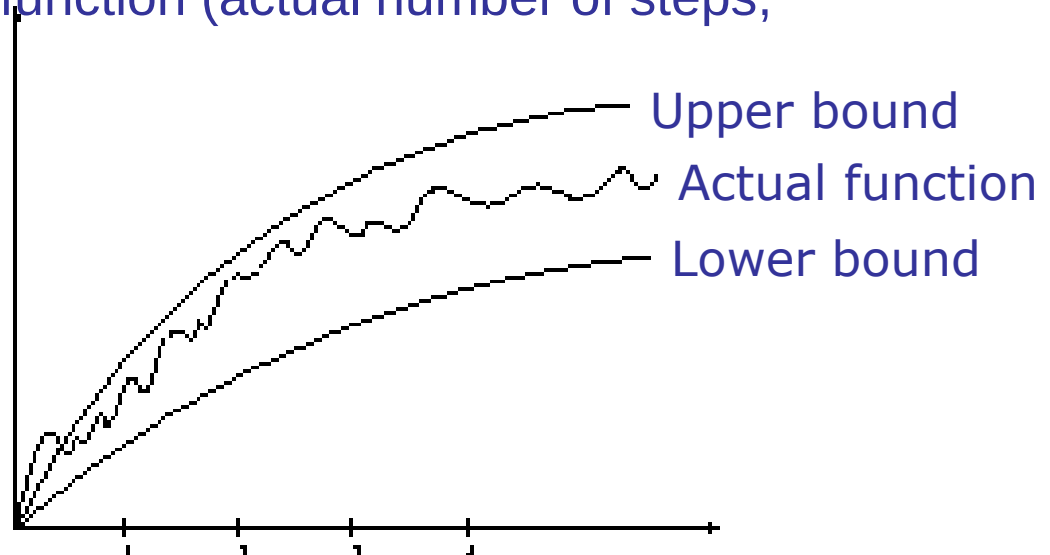
- **Worst Case Complexity:**
 - the function defined by the *maximum* number of steps taken on any instance of size n
- **Best Case Complexity:**
 - the function defined by the *minimum* number of steps taken on any instance of size n
- **Average Case Complexity:**
 - the function defined by the *average* number of steps taken on any instance of size n

Best, Worst, and Average Case Complexity



Doing the Analysis

- It's hard to estimate the running time exactly
 - Best case depends on the input
 - Average case is difficult to compute
 - So we usually focus on worst case analysis
 - Easier to compute
 - Usually close to the actual running time
- Strategy: find a function (an equation) that, for large n , is an upper bound to the actual function (actual number of steps, memory usage, etc.)



Classifying functions by their Asymptotic Growth Rates (1/2)

- asymptotic growth rate, asymptotic order, or order of functions
 - Comparing and classifying functions that ignores
 - *constant factors* and
 - *small inputs*.
- The Sets big oh $O(g)$, big theta $\Theta(g)$, big omega $\Omega(g)$

Classifying functions by their Asymptotic Growth Rates (2/2)

- $O(g(n))$, Big-Oh of g of n , the Asymptotic Upper Bound;
- $\Theta(g(n))$, Theta of g of n , the Asymptotic Tight Bound; and
- $\Omega(g(n))$, Omega of g of n , the Asymptotic Lower Bound.